

WiseCash Social Commerce Marketplace

Product & Build Spec — Pivot Concept

1. Vision

WiseCash becomes a phone-number-native social commerce app: WhatsApp's trust model (you must have someone's number to reach them) combined with Instagram's public discovery feed (Reels/Photos), plus a full POS/Inventory/Invoicing engine running directly inside the chat thread. Every seller's sales and every buyer's purchases roll up automatically into a business Dashboard.

Nobody in the Tanzanian/East African market currently combines "chat you already trust" with "commerce you can complete without leaving the conversation." That gap is the opportunity.

2. Core Concept (clarified)

Layer	Who sees it	Purpose
Update Feed (Reels + Photos)	Public — anyone on the app	Discovery, marketing, brand building
Chat	Only mutual contacts who've approved a follow request	Trust-based conversations, real deals
Commerce-in-chat (sell, buy, invoice)	Only inside an approved chat thread	Every transaction has an identity behind it
Dashboard	Only the business owner/staff	POS, inventory, sales & purchase summary, rep

Identity model:

- Registration = phone number (OTP verification), same trust root as WhatsApp.
- Every user also sets a Username — this becomes their public Profile ID (like an Instagram handle), shown on their Update posts instead of exposing their phone number publicly.
- Chat eligibility = User A has User B's number saved AND B has approved A's follow request (mutual, two-sided — like a stricter version of WhatsApp contact sync + Instagram private-follow).

3. Feature Breakdown

3.1 Onboarding & Identity

- Phone number + OTP registration (Africa's Talking, Twilio Verify, or ClickPesa's SMS gateway if bundled).
- Username selection (unique, becomes Profile ID / public handle).
- Optional: business profile setup (shop name, category, logo) — flags the account as a Seller Account with Dashboard access.

3.2 Update Feed (public)

- Post Photos and Reels (short vertical video).
- Public feed, algorithmic or chronological (start chronological — cheaper to build, add ranking later).
- Like/comment/share on posts.

- "Shop this post" button — if the poster is a Seller Account, tapping it opens a chat thread with them (this is the discovery-to-chat bridge).

3.3 Follow / Chat Access

- Follow request sent by Username or phone number.
- Approval required both ways before chat unlocks (prevents spam and protects sellers from unsolicited buyers cluttering their business inbox).
- Once approved: standard 1:1 chat (text, image, voice notes, read receipts).

3.4 Commerce-in-Chat

- Inline product catalog picker — seller attaches a product from their inventory directly into the chat as a rich card (photo, price, stock status).
- Buyer requests quantity, seller confirms, and a Generate Invoice button creates an invoice object inside the thread (PDF or in-app rich card).
- Payment integrates ClickPesa Checkout Link (mobile money — M-Pesa, Tigo Pesa, Airtel Money, Halopesa) directly in the invoice card.
- Order status updates (Pending, Paid, Fulfilled) update live in the chat for both parties.
- Supplier purchasing uses the same mechanism in reverse — a shop owner chats with their supplier's Seller Account and places a purchase order the same way. This feeds the Purchases side of the Dashboard.

3.5 Dashboard (Seller Accounts only)

- Sales summary (daily/weekly/monthly), by product, by buyer.
- Purchases summary (stock bought from suppliers).
- Inventory levels (auto-decrement on sale, auto-increment on confirmed purchase).
- Staff/team access (multi-user per business, permission levels).
- This is effectively the existing wisecash.app POS/Inventory core, re-plumbed to receive its transactions from chat instead of (or in addition to) a standalone POS screen.

3.6 Settings ("Tools" button)

- Account, privacy, notification settings.
- Business settings (tax rate, currency, receipt template, ClickPesa account linking).
- Blocked/reported users.

3.7 AI Assistant

Practical, scoped uses (avoid trying to do everything at once):

- For buyers: "Find me a phone charger under 10,000 TSh" — searches the Update Feed and connected suppliers, surfaces matching posts/sellers.
- For sellers: auto-draft product descriptions from a photo; auto-suggest reorder when stock is low; summarize weekly sales in plain language.

- In chat: smart replies for common seller questions ("Do you have this in stock?") drafted from live inventory data.
- Keep AI assistive, not autonomous, for money-moving actions — a human always confirms before an invoice is sent or paid.

4. Data Model (high-level)

- User — id, phone_number (unique, verified), username (unique), account_type (personal/seller), profile (display_name, avatar, bio)
- Contact / FollowEdge — requester_id, target_id, status (pending/approved/blocked). Chat unlocks only when BOTH directions = approved.
- Post (Update) — author_id, type (photo/reel), media_url, caption, visibility (always public, per clarified model)
- ChatThread — participant_a_id, participant_b_id — requires mutual FollowEdge = approved
- Message — thread_id, sender_id, type (text/image/product_card/invoice_card), payload
- Product — owner_id (seller), name, price, stock_qty, photo, category
- Order / Invoice — thread_id, buyer_id, seller_id, line_items[], total, status, payment_ref (ClickPesa)
- Transaction (Dashboard rollup) — type (sale/purchase), order_id, amount, timestamp

5. Recommended Tech Stack — Python-First Backend

Python is not just a convenient shared language here — it carries the biggest and most critical parts of the server side. The core API, the real-time chat engine, every background job (invoice generation, notification fan-out, inventory sync, video transcoding), the entire AI assistant layer, and all Dashboard analytics run on Python. Only the thin edges of the system — the mobile client (Flutter) and raw object storage (S3/R2) — sit outside it. This also means a small team can move fast, hire easily in the TZ/EA market (Python is widely taught), and reuse the same libraries across every major subsystem.

Where Python carries the load:

- Business logic and API — every request (auth, follow/chat approval, catalog, invoicing, payments) is handled by Python.
- Real-time messaging — chat delivery, presence, and read-receipts run in Python's async layer, not offloaded to a separate service.
- All background processing — Celery workers handle everything that happens after a request: sending invoices, syncing stock, compressing Reels, pushing notifications.
- The entire AI assistant — prompt construction, tool calls, and streaming responses are native Python, with no bridging layer to another language.
- Dashboard intelligence — sales/purchase rollups, low-stock alerts, and weekly summaries are computed in Python (pandas) before ever reaching the app.

5.1 Core Backend

Component	Python Choice	Why
API framework	FastAPI	Async-native, very fast, automatic OpenAPI docs — ideal for a
ORM / DB layer	SQLModel or SQLAlchemy 2.0	Type-safe, integrates cleanly with FastAPI and Pydantic validation
Admin/back-office	Django (optional, separate service)	If internal staff need a quick admin panel for support/moderation
Real-time chat	FastAPI WebSockets or Django Channels	Python-backed by Redis pub/sub, presence, and read-receipts d

Component	Python Choice	Why
Background jobs	Celery + Redis or RabbitMQ	Invoice generation, notification fan-out, inventory sync, and video processing
Media processing	Pillow (photos), moviepy / ffmpeg-python (video transcoding)	Keep photo transcoding and video compression inside the same Python process
AI assistant	Anthropic Python SDK, called directly from FastAPI	Same FastAPI handles the rest of the backend — no cross-language glue
Data & Dashboard analytics	Plotly/Matplotlib (server-rendered charts) or roll-up libraries like Superset	See charts, use roll-ups, investigate alerts, and make AI summaries

5.2 Supporting Infrastructure

Layer	Recommendation	Why
Mobile app (iOS + Android)	Flutter	Single codebase, near-native performance, strong media widget support
Web app	Django or FastAPI serving a lightweight React Dashboard	Keep Dashboard near, is only Python making SaaS adding glue
Database	PostgreSQL (core data) + Redis (chat, product, cache)	Standard, reliable, both have first-class Python drivers
Media storage	Cloudflare R2 or S3	Object storage for processed photos/video; Python (boto3) handles uploads
Payments	ClickPesa Checkout Link API, called from Python server	Already licensed in Kenya, supports all major TZ mobile money
Push notifications	Firebase Cloud Messaging, triggered from Celery tasks	Firebase is the standard, easy to call from Python via firebase-admin
Deployment	Docker + Gunicorn/Uvicorn workers behind Nginx	Simplifies DevOps for Python + FastAPI, no Celery

6. Suggested Build Order (MVP to v2)

Phase 1 (MVP, roughly 6–8 weeks with a small team):

Phone auth, Username, basic chat (approved contacts only), product card in chat, manual invoice, ClickPesa payment link, basic Dashboard (sales list).

Phase 2:

Public Update Feed (photos first, Reels later — video is expensive), "Shop this post" bridge, follow requests, purchase-from-supplier flow, inventory auto-sync.

Phase 3:

Reels/video, AI assistant (product description generation + smart replies), advanced Dashboard analytics, staff/multi-user accounts.

Phase 4:

Ratings/reviews for sellers, ads/promoted posts as a monetization layer.

7. Monetization & Profit Model

The goal is a model where users get enough value to stay and transact, while every layer of activity — chatting, selling, buying, discovering — creates a path to revenue for the platform owner. Below are the revenue streams that make the most sense for this product, followed by what keeps users willing to pay.

7.1 Revenue Streams for the Owner

Stream	How it works	Why it fits this product
Transaction commission	A small percentage (e.g. 1-3%) on every in-	High sales volume, so this small fee platform's revenue grows
Seller subscription tiers	Free tier (basic catalog, limited monthly orders); Paid Business tier (unlimited orders, product of Das	Product/Business tier revenue (MRR) is predictable, steady, and repeatable
Promoted Updates / Feeds	Sellers pay to boost a Reel/Photo post so it reaches more of the public feed; an initial budget can be	Public feed is the top of the funnel; each Reel/Photo post is a potential lead
Marketplace supplier fees	Verified suppliers pay a small listing or featured placement fee to appear prominently in high-tailor sa	Retailer-to-supplier purchasing is a high-margin, high-volume business
Premium AI features	Free tier gets basic AI replies; Pro tier unlocks AI-generated product descriptions, autogenerated sugg	AI-generated product descriptions, autogenerated suggestions, and other AI features are a high-margin, high-volume business
Payment gateway referral	A small negotiated margin on top of the ClickBank processing fee; any payment through a company with the	Every transaction is a potential referral; any payment through a company with the
Aggregated market insight	Anonymous, aggregated sales trend reports (e.g. product listing placement data that has Dealers, Sales	Every product listing placement is a potential lead; aggregated data is a high-margin, high-volume business
Add-on services	Optional paid onboarding/training for new sellers, effort to increase brand awareness, or other services	Effort to increase brand awareness, or other services are a high-margin, high-volume business

7.2 What Keeps Users Willing to Pay / Transact

- Buyers: free to browse, follow, and chat — friction-free discovery is what pulls them in and keeps the feed active.
- Sellers (free tier): enough functionality to run a small shop for free builds trust and adoption — this is the top of the funnel.
- Sellers (paid tier): once volume grows, the Dashboard, multi-staff access, and advanced analytics become genuinely necessary — the upgrade decision makes itself.
- Suppliers: paid placement only makes sense once there's a real audience of retailers actively purchasing through the app — sequence this after Phase 2 traction, not at launch.

7.3 Suggested Sequencing for the Owner

- Launch: transaction commission only (simplest, lowest barrier to adoption, revenue starts from day one of real usage).
- Once there's a meaningful seller base: introduce the Free vs Pro subscription tiers, priced against the value of the advanced Dashboard.
- Once the feed has real daily engagement: introduce Promoted Updates as an ad product.
- Once retailer-to-supplier purchasing volume is proven: introduce supplier listing/placement fees.
- Ongoing, low-effort: aggregated market insight reports as a side revenue line once transaction data reaches meaningful scale.

7.4 How This Compares to Established Social Platforms

The revenue model above is not new — it mirrors patterns already proven at scale by the largest social platforms. Seeing WiseCash's streams mapped against theirs is useful validation that this approach works.

Platform Pattern	How They Do It	WiseCash Equivalent
Advertising (Meta, TikTok, Instagram)	The feed is the product — businesses pay to reach users via sponsored posts, Reels, or Photos	Promoted Updates / Feeds

Platform Pattern	How They Do It	WiseCash Equivalent
Transaction commission (Instagram/TikTok Shop, Facebook Marketplace)	A percentage cut (typically 2-8%) taken on sales completed through the platform's marketplace	Seller commission on sales completed through the platform's marketplace
Subscriptions for power users (LinkedIn Premium, Snapchat+, X Premium)	Free for everyone; a paid tier unlocks analytics, priority visibility, and other perks for people who do a lot of work on the platform	Seller Priority visibility and analytics dashboard for people who do a lot of work on the platform
Verified / priority placement (LinkedIn, X, TikTok)	Paying to be seen first — verified badges, sponsored posts, and other perks	Supplier listing and featured placement fees
Data and insights (Meta, LinkedIn)	Aggregated, anonymized trend data sold to advertisers and other businesses	Aggregated market insight reports (e.g. top-selling products)
Platform-as-infrastructure fees (WhatsApp Business API, Messenger Platform)	Businesses pay per message or per API call to use the platform's infrastructure	SMS/notification credits from Meta, by using the platform's infrastructure

The pattern across all of them: free for individual users, paid for anyone using the platform to run or grow a business. WiseCash already mirrors this — buyers stay free forever, sellers eventually pay because the platform becomes core infrastructure for their shop.

7.5 Deep Dive: The TikTok Model, Mapped to WiseCash

TikTok is a useful reference because it combines four revenue mechanics on top of a free public feed, and each one has a direct WiseCash equivalent already in this plan.

TikTok Mechanic	How TikTok Runs It	WiseCash Equivalent
Advertising (the majority of revenue)	Branded content, TopView placements, and brand takeover ads	Promoted Updates: businesses pay for each video in the feed
Shop commission	TikTok Shop takes a commission on every sale completed in-app, plus a separate advertising fee for product placement	Transaction commission on sales completed in-app, plus a separate advertising fee for product placement
Virtual gifting / coins	Users buy Coins with real money, then send virtual gifts to creators and paying 'besties'	Optional gifts to creators and paying 'besties'; cheap platform fee
Creator payouts (cost, not revenue)	TikTok pays qualifying creators through the Creator Rewards Program, funded by the ad revenue	Creator Rewards Program, funded by the ad revenue

TikTok's own numbers show why advertising leads: roughly 77% of TikTok's revenue comes from advertising and about 23% from commerce and in-app purchases combined, with TikTok Shop earning through a mix of transaction commissions, seller advertising fees, and promoted product placements. The lesson for WiseCash: the public Update Feed should be treated as the primary long-term revenue engine (via Promoted Updates), while transaction commission is the fastest stream to switch on at launch.

8. Growth Strategy — Path to Scale

Reaching a genuinely large user base is less about adding features and more about which growth levers get pulled, and in what order. Each lever below is marked by what it actually requires — some are pure product work that can start immediately, others need a person or an ops process behind them, and a couple are later-stage moves that only make sense once real scale exists.

8.1 Viral Loop — Buildable Now

- Referral tracking: every user gets a unique referral link; a `referred_by` field on signup is enough to track it.
- "Invite 3, unlock Pro free for a month": a background job (Celery) checks referral counts daily and auto-upgrades qualifying accounts.
- "Share my shop" button: generates a pre-filled deep link and opens the native share sheet targeting WhatsApp/Instagram directly — a standard mobile SDK call, not a partnership.
- The follow-request-requires-both-numbers model already mirrors how WhatsApp grew organically — people join because someone they know is already there.

8.2 Sellers as an Unpaid Marketing Force — Buildable Now

- Auto-generate a branded overlay ("Shop on WiseCash — @username") on any Reel/Photo a seller exports, using the same Python media pipeline already in the stack (Pillow/ffmpeg).
- A "Download for TikTok/Instagram" button on product cards exports a pre-branded version, making sellers' existing posting habits faster while planting the brand in their external content.

8.3 Low-Bandwidth & Feature-Phone Support

Lite mode (buildable now): compressed video previews by default, full quality only on WiFi or on tap; a stripped-down build with fewer animations and a text-first feed — pure engineering, no partnerships needed.

USSD support for feature phones (needs a vendor, not a build-alone effort): USSD runs through an aggregator such as Africa's Talking, Beem Africa, or Selcom, who already hold the telecom relationships and lease a short code. A menu tree ("1. Check orders 2. Confirm sale 3. Send invoice via SMS") sits behind their API — an integration project, not a negotiation with telecoms directly.

8.4 Telecom Zero-Rating & Mobile Money Partnerships — Later-Stage

Zero-rating requires a telecom to whitelist app traffic at the network level, which is a business negotiation, not something engineering can deliver. Telecoms typically only do this for apps that already bring them scale or transaction revenue. The realistic path: since payments likely run over mobile money rails (Tigo Pesa, M-Pesa, Airtel Money via ClickPesa), telecoms already profit from WiseCash's growth without zero-rating — a real pitch, but one to make once there is meaningful user or transaction volume to offer in return. Telecom-backed startup accelerators (Vodacom, Airtel, CRDB all run these in Tanzania) are typically a more attainable near-term door than a cold zero-rating ask.

8.5 Daily Engagement Beyond Commerce

Engineering (straightforward): a trending feed ranked by likes, comments, and shares over a rolling 24-48 hour window, weighted by recency — a SQL query with a decay function, not exotic machine

learning.

Operations (the harder part): a cold-start feed with few users has little to rank, so a manually curated "Local Market Trending" carousel is needed in the early months — a content/community job until organic volume is sufficient.

8.6 Full Swahili-First Localization

- Engineering setup (standard i18n framework, e.g. Flutter's intl package) is a day's work, not the hard part.
- The real effort is writing every string, error message, invoice label, and AI assistant response in Swahili first, with English as a fallback — translated-after-the-fact apps consistently feel foreign; Swahili-native ones don't.
- Extends to formats people already use locally — how TSh amounts are written, date formats, and payment terminology matching what mobile money users already recognize.

8.7 Trust at Scale

- Engineering: a verified flag and badge on Seller Accounts (via phone verification, business registration upload, or manual review), plus a simple 1-5 star rating tied to completed orders.
- Operations: verification review and dispute handling don't scale automatically — start manual with a small internal team, and only automate once real data exists on what disputes actually look like.

8.8 API / Ecosystem Layer — Deliberately Last

Technically this is exposing versioned REST endpoints (FastAPI makes this natural, since most of the internal API likely already exists) with API keys or OAuth for third-party developers. The real requirement is documentation, rate limiting, and a reason for outside developers to build against the platform — which only exists once there is real user and transaction volume to plug into. This is a later-stage move, not a launch feature.

8.9 Overall Sequencing

- Start immediately: viral loop, seller-as-marketer tools, lite mode, Swahili-first localization — all pure product work with no external dependency.
- Needs a person, not just code, from early on: trending feed curation, seller verification and dispute review.
- Needs a vendor integration: USSD support via a licensed aggregator.
- Later-stage, once scale or transaction volume exists: telecom zero-rating conversations, API/ecosystem layer.